

I. Program Documentation

1. Introduction

2. Scheduler in GPU

3. Instruction Fetcher Module

4. Instruction Decoder Module

5. Arithmetic Logic Unit (ALU)

II. Testbench Documentation

1. Verification Methodology

2. Module-Level Testbenches

3. GPU Simulation Using Cocotb

4. Simulation Result

III. Contribution

IV. Acknowledgements

V. References

I. Program Documentation

Introduction

Graphics Processing Units (GPUs) are computing devices designed to efficiently process large amounts of data in parallel. They are particularly well suited for workloads that require many operations to be executed simultaneously. Although GPUs were originally developed for graphics rendering, their application has expanded significantly to domains such as machine learning, scientific computing, and big data processing.

In contrast to Central Processing Units (CPUs), which primarily execute instructions sequentially with a limited number of powerful cores, GPUs employ a highly parallel architecture consisting of many lightweight processing cores operating concurrently. This architectural difference allows GPUs to achieve significantly higher throughput when handling data-parallel tasks, making them more efficient than CPUs for suitable workloads.

The project presented in this report, titled Small GPU, focuses on the design, modeling, and simulation of a simplified GPU architecture. The primary objective of the project is to understand the fundamental hardware components that enable parallel execution in GPUs by implementing them at the register-transfer level using Verilog. Rather than optimizing for maximum performance, the design emphasizes clarity, modularity, and correctness to support educational exploration.

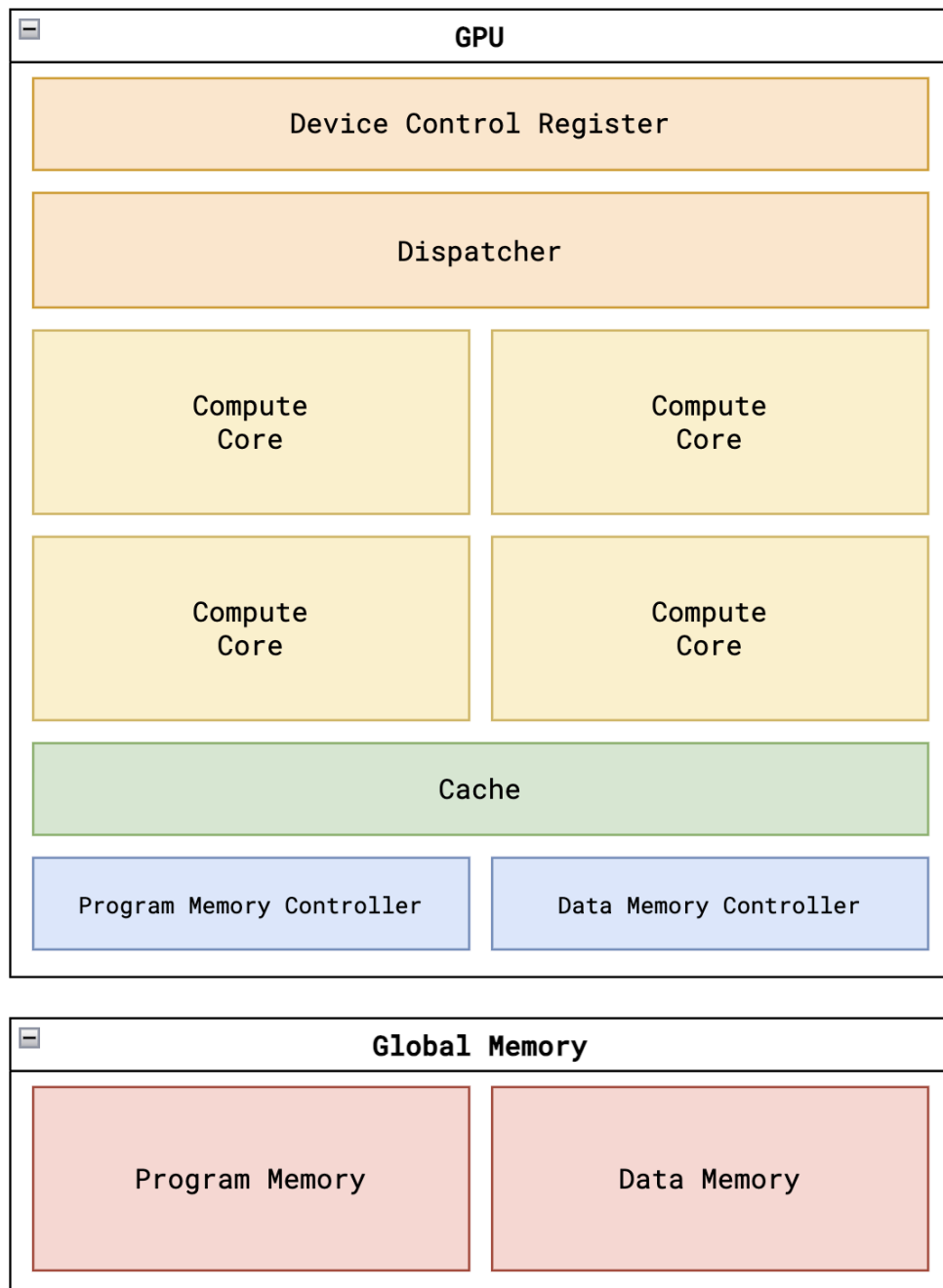
The main problem addressed in this project is the design and integration of core GPU hardware components that support parallel instruction execution and multithreading. These components include the Arithmetic Logic Unit (ALU), instruction decoder, fetch unit, scheduler, compute core, and global memory interface. Each module is designed, modeled, and simulated to demonstrate its role within the overall GPU architecture and its interaction with other components.

The system requirements for the Small GPU project include support for multiple concurrent threads, a shared global memory space, basic arithmetic and memory instructions, and a scheduling mechanism to manage thread execution. The scheduler is required to ensure fair and starvation-free execution of threads, while the compute core must correctly execute instructions fetched from memory. The design must be synthesizable, modular, and verifiable through simulation and test benches.

Through the implementation and simulation of these components, this project provides insight into how GPUs achieve parallelism and high throughput, serving as a foundational study of GPU architecture and hardware design principles.

1.1 GPU Architecture

The design of a GPU focuses on supporting large-scale parallel processing and high computational bandwidth. This is accomplished through the integration of many functional components that work together to execute operations in an efficient manner.



- **Device Control Register:** This unit is responsible for controlling and configuring the operational state of the GPU. It enables the host system to manage GPU execution and monitor its status.
- **Dispatcher:** The dispatcher distributes tasks among the Compute Cores. Its role is to balance workloads across the architecture in order to improve performance and reduce idle core time.
- **Cache:** The cache functions as a fast local storage layer within the GPU architecture, temporarily holding commonly used instructions and data. Its primary purpose is to shorten data retrieval time by reducing dependence on slower main memory.
- **Program Memory Controller and Data Memory Controller:** These units are responsible for regulating instruction and data movement between the compute resources and memory subsystems. By managing memory transactions and synchronizing read/write operations, they

help maintain steady data throughput and prevent performance degradation caused by memory access delays.

1.2 Global Memory

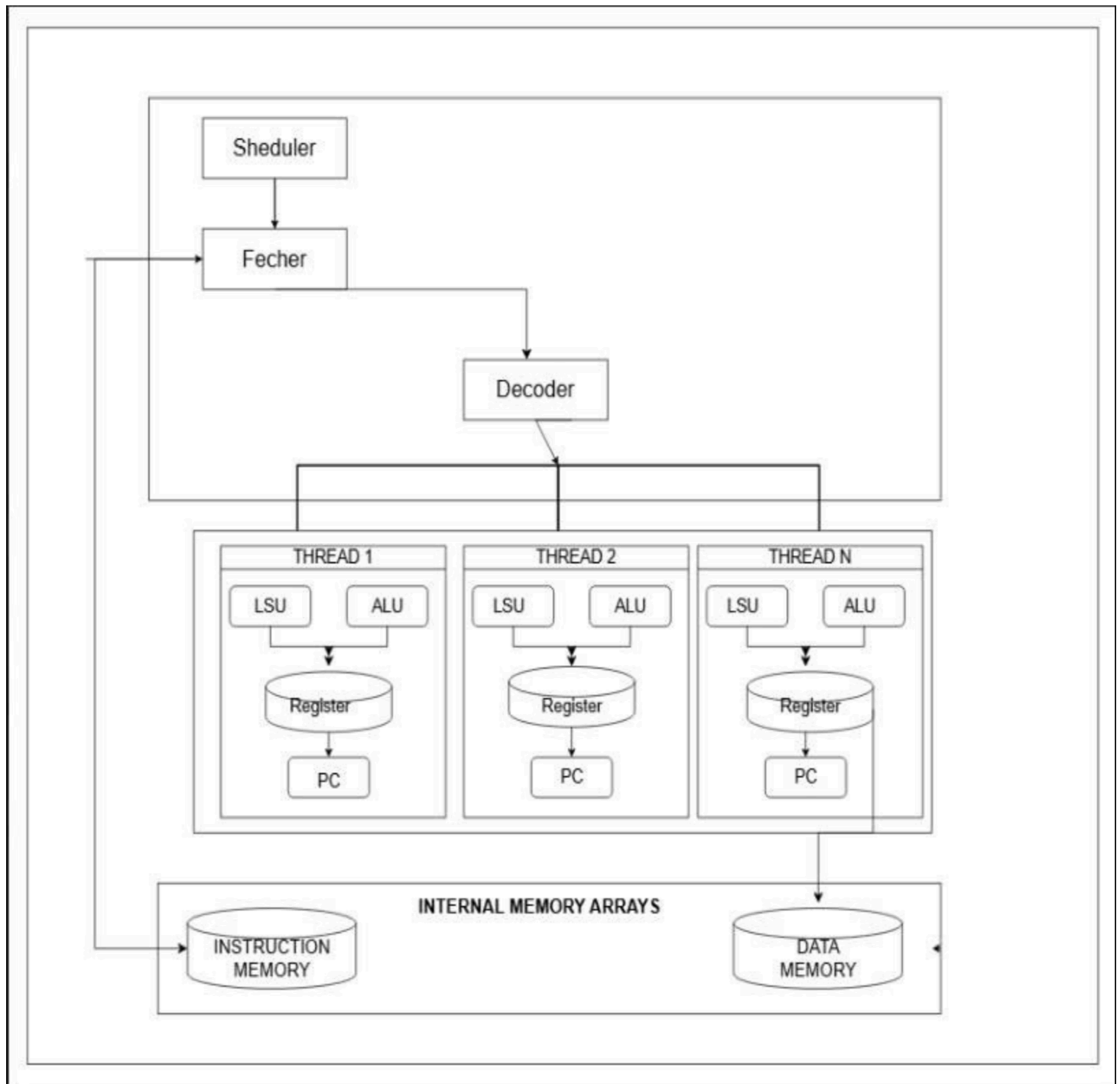
Global memory is a shared memory component in GPU architectures that provides a common address space accessible by all threads. Compared to registers or local memory, global memory offers larger storage capacity but with higher access latency.

The design of global memory prioritizes simplicity, correctness, and educational clarity rather than performance optimization. All threads access a unified global memory space, and key parameters such as data width and address width are centrally defined to ensure consistency across modules. This design approach improves maintainability, scalability, and ease of integration with other components such as the compute core and scheduler.

To improve clarity and efficiency in the memory access pipeline, global memory in the Small GPU is logically separated into Program Memory and Data Memory. Program Memory is responsible for storing the instructions executed by the GPU. These instructions are fetched by the instruction fetch unit, decoded, and then dispatched to the compute core. Since program instructions are generally read-only during execution, program memory is optimized for sequential and predictable access patterns, simplifying control logic and reducing potential hazards.

Data Memory, in contrast, stores the input data required for computation as well as the output results produced by executing threads. It is accessed through load (LDR) and store (STR) instructions issued by the compute core. Unlike program memory, data memory must support both read and write operations and handle more irregular access patterns due to concurrent access by multiple threads. This separation between program and data memory reflects common architectural practices and improves conceptual clarity when designing and debugging the system.

1.3 Core Compute



- **Compute Core:** The compute core is the central execution unit of the GPU, responsible for coordinating instruction processing, thread execution, and data movement across all internal components.
- **Scheduler:** The scheduler selects the next active thread to execute, ensuring fair and controlled execution among multiple threads within the compute core.
- **Fetcher:** The fetcher retrieves instructions from instruction memory based on the program counter (PC) of the scheduled thread.

- **Decoder:** The decoder interprets fetched instructions by extracting opcode, register indices, and immediate values required for execution.
- **Arithmetic Logic Unit (ALU):** The ALU performs arithmetic and comparison operations such as addition, subtraction, multiplication, and condition evaluation.
- **Load/Store Unit (LSU):** The LSU handles memory access operations, enabling data transfer between registers and global or shared memory.
- **Program Counter (PC):** Each thread maintains its own program counter, tracking the current instruction address and supporting control flow changes such as jumps.
- **Register File:** Each thread is associated with a private register file that stores operands and computation results locally.
- **Multi-thread Execution Structure:** The compute core supports multiple threads executing the same instruction flow independently, sharing control logic while maintaining separate execution states.

1.4 Key Features of GPU Architecture

The GPU architecture is characterized by several key features that enable high computational performance:

1. **Massive Parallelism:**

GPUs are composed of a large number of lightweight processing cores that execute thousands of threads concurrently. This design supports data-parallel workloads and follows a SIMD/SIMT execution model, allowing the same instruction to be applied to multiple data elements simultaneously.

2. **Scalability:**

The modular organization of GPU architecture allows performance to scale by increasing the number of Compute Cores and execution units. This scalability makes GPUs adaptable to a wide range of performance requirements and application domains.

3. **Energy Efficiency:**

GPUs are designed to maximize computational throughput per watt of power consumed. By executing many parallel operations with simple cores and efficient scheduling, GPUs achieve higher energy efficiency compared to general-purpose processors for parallel workloads.

4. **Thread Management and Scheduling:**

Thread execution is efficiently controlled through hardware-based scheduling mechanisms. The Scheduler and Dispatcher coordinate thread allocation and context switching, enabling effective workload distribution and hiding memory latency by switching between active threads.

5. **Memory Hierarchy and Bandwidth:**

GPUs employ a hierarchical memory system, including registers, cache, and global memory, to

optimize data access. High memory bandwidth and fast on-chip storage reduce access latency and support sustained throughput for memory-intensive applications.

Modules

2. Scheduler in GPU

2.1 Concept of the Scheduler

In parallel processing architectures such as Graphics Processing Units (GPUs), multiple threads are executed concurrently to improve computational throughput. However, because the number of execution units is limited, not all threads can be executed simultaneously in every clock cycle. Therefore, a scheduler is required to manage the execution order of threads and allocate computational resources efficiently.

In the Small GPU project, the scheduler is designed to select exactly one active thread per clock cycle and dispatch it to the compute core. The scheduler follows a Round Robin scheduling policy, in which threads are executed in a cyclic order. This approach ensures fairness by giving each active thread an equal opportunity to execute and prevents starvation. Additionally, the scheduler supports interleaved multithreading, allowing the GPU to hide latency caused by memory access or long-latency operations, thereby improving overall system throughput.

2.2 Verilog Implementation of the Scheduler

The scheduler is implemented as a Verilog module and relies on shared architectural parameters defined in `definitions.vh`, such as the number of threads and the width of thread identifiers. The complete implementation of the scheduler module is shown below.

```

`include "definitions.vh"

module scheduler #(
    parameter NUM_THREADS = `NUM_THREADS
)(
    input clk,
    input reset,
    input [NUM_THREADS-1:0] active_threads,
    output reg [`THREAD_ID_WIDTH-1:0] scheduled_thread
);
    integer last_thread;
    integer i;
    integer thread;
    reg thread_found;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            last_thread <= -1;
            scheduled_thread <= 0;
        end else begin
            thread_found = 0;

            // Iterate through all threads using Round Robin
            for (i = 1; i <= NUM_THREADS; i = i + 1) begin
                thread = (last_thread + i) % NUM_THREADS;

                // Select the first active thread found
                if (active_threads[thread] && !thread_found) begin
                    scheduled_thread <= thread;
                    last_thread <= thread;
                    thread_found = 1;
                end
            end
        end
    end
endmodule

```

2.3 Explanation of the Scheduler Operation

To clearly describe the behavior of the scheduler, this section is organized into inputs, outputs, and internal operation, following the logical data flow of the module.

2.3.1 Inputs

The scheduler receives three input signals: `clk`, `reset`, and `active_threads`.

The `clk` signal is the system clock that determines when scheduling decisions are made. The scheduler updates its internal state and selects a new thread only on the rising edge of the clock, ensuring synchronous and deterministic behavior.

The `reset` signal initializes the scheduler to a known state. When `reset` is asserted, the scheduler clears its internal state by setting the previously scheduled thread to an invalid value and initializing the output thread ID. This guarantees correct behavior during system startup or reset conditions.

The `active_threads` signal is a bitmask representing the execution status of all threads. Each bit corresponds to one thread: a value of 1 indicates that the thread is active and eligible for scheduling, while a value of 0 indicates that the thread has halted or completed execution. This input prevents inactive threads from being scheduled.

2.3.2 Output

The scheduler produces a single output signal, `scheduled_thread`.

The `scheduled_thread` output specifies the ID of the thread selected for execution in the current clock cycle. This output is forwarded to the compute core, where it controls instruction fetch, decode, and execution for the selected thread. By generating exactly one thread ID per cycle, the scheduler enforces controlled and orderly execution within the GPU pipeline.

2.3.3 Internal Operation and Scheduling Logic

Internally, the scheduler implements a Round Robin scheduling algorithm. The variable `last_thread` stores the ID of the thread that was scheduled in the previous clock cycle. This variable enables the scheduler to continue selection from the next thread in sequence, rather than restarting from the first thread each time.

Upon reset, `last_thread` is initialized to -1, allowing the scheduler to begin scheduling from thread 0 during the first active cycle. During normal operation, the scheduling logic executes inside an always block triggered on the rising edge of the clock.

At each cycle, the scheduler iterates through all threads using a for loop. The expression $(\text{last_thread} + i) \% \text{NUM_THREADS}$ ensures a circular search order, which is the core mechanism of the Round Robin policy. The scheduler checks each candidate thread's corresponding bit in `active_threads` to determine whether it is active.

The control flag `thread_found` ensures that only one thread is selected per clock cycle. Once an active thread is found, the scheduler updates `scheduled_thread`, records the selected thread in `last_thread`, and prevents further selections in the same cycle. This mechanism guarantees fairness, prevents starvation, and maintains low hardware complexity.

2.4 Results and Observations

Simulation results show that the scheduler correctly implements Round Robin scheduling. All active threads are executed in a cyclic order, and no thread experiences starvation as long as it remains active. When some threads stall due to memory access latency, other active threads continue to be scheduled, demonstrating effective interleaved multithreading.

However, because the scheduler does not support priority-based execution, all threads are treated equally regardless of workload characteristics. In scenarios with unbalanced workloads or latency-sensitive tasks, this may result in suboptimal performance. Despite this limitation, the simplicity, fairness, and low hardware overhead of the design make it well suited for the Small GPU project and educational purposes.

3. Instruction Fetcher Module

3.1 Instruction Fetch Concept and Mechanism

The instruction fetcher is responsible for retrieving instructions from instruction memory based on the current program counter value. In this simple GPU design, the fetcher does not perform any computation or control decisions; instead, it acts as a read-only interface between the compute core and the instruction memory. At each execution cycle, the fetcher receives a program counter value from the compute core and outputs the corresponding instruction. This design keeps the fetch stage simple and allows instruction sequencing to be fully controlled by the compute core logic.

3.2 Program Counter Management

The fetcher does not manage or update the program counter internally. The program counter is maintained by the compute core on a per-thread basis and reflects the current execution position of each thread. The fetcher only uses the provided program counter value to access instruction memory. Any updates to the program counter, such as sequential execution or branching, are handled outside the fetcher, ensuring a clear separation between instruction fetching and control flow logic.

3.3 Fetcher Source Code

```
`include "definitions.vh"

module fetcher (
    input [3:0] pc_in,  // 4-bit PC for 16 entries
    output [`DATA_WIDTH-1:0] instruction
);
    // Instruction memory - 16 entries
    reg [`DATA_WIDTH-1:0] instr_mem [0:15];

    integer i;  // Declare at module level

    // Non-pipelined fetch - combinational read
    assign instruction = instr_mem[pc_in];

    // Initialize instruction memory
    initial begin
        $readmemh("src/instruction_memory.mem", instr_mem);
        $display("Instruction Memory Initialized in Fetcher:");
        for (i = 0; i < 16; i = i + 1) begin
            $display("instr_mem[%0d] = %h", i, instr_mem[i]);
        end
    end

endmodule
```

3.4 Code Explanation

The fetcher module reads an instruction from instruction memory based on the current program counter value. Instruction memory is implemented as a small register array and is accessed using combinational logic, allowing the instruction output to change immediately when the program counter changes. The instruction memory is initialized from an external file at simulation start, enabling flexible program loading without modifying the source code. Overall, the fetcher contains no control or state logic and relies entirely on the compute core to manage instruction flow.

3.5 Result Observation

The simulation output shows the sequence of fetched instructions corresponding to the program counter values, confirming that the fetcher correctly retrieves instructions from instruction memory. The printed instruction contents verify that the memory is initialized properly and that instruction addressing

works as expected. This observation demonstrates that the fetch stage functions correctly as the entry point of the compute core execution pipeline.

4. Instruction Decoder Module

4.1 Decoder Concept

The Instruction Decoder is a critical component in the GPU execution pipeline. Its primary role is to interpret binary instructions fetched from instruction memory and convert them into structured control signals that can be used by subsequent execution units. By decomposing each instruction into well-defined fields, the decoder ensures that the GPU correctly identifies the operation to perform and the operands involved.

4.2 Decoder Functional Overview

During instruction execution, the decoder operates immediately after the fetch stage. It receives a 16-bit instruction word and extracts the essential fields required for execution, including the opcode, register indices, and immediate values. This decoding process forms the foundation for correct instruction interpretation and control flow within the GPU compute core.

4.3 Instruction Format

In this design, each instruction is 16 bits wide and follows a fixed-format encoding. The instruction is divided into several fields as follows:

- Opcode (bits [15:12]): Specifies the operation to be executed.
- Destination register (bits [11:8]): Indicates the register where the result will be written.
- Source register 1 (bits [7:4]): First operand register for register-based (R-type) instructions.
- Source register 2 (bits [3:0]): Second operand register for R-type instructions.
- Immediate value (bits [7:0]): Constant operand used in immediate-based (I-type) instructions.

Although both R-type and I-type fields are extracted simultaneously, only the relevant fields are utilized depending on the instruction type.

4.4 Decoder Implementation

```
// decoder.v
`include "definitions.vh"

module decoder (
    input [`DATA_WIDTH-1:0] instruction,
    output reg [3:0] opcode,
    output reg [3:0] dest_reg,
    output reg [3:0] src1_reg,
    output reg [3:0] src2_reg,
    output reg [7:0] immediate
);
    always @(*) begin
        opcode      = instruction[15:12];
        dest_reg    = instruction[11:8];
        src1_reg    = instruction[7:4];
        src2_reg    = instruction[3:0];
        immediate   = instruction[7:0];
    end
endmodule
```

The decoder is implemented as pure combinational logic using an always @(*) block. Each output signal is directly derived from fixed bit positions within the instruction word. This design ensures deterministic and cycle-independent decoding behavior without introducing internal state or pipeline latency.

4.5 Decoder Output Signals

The decoder produces structured control outputs that drive subsequent GPU modules:

- opcode determines the operation executed by the ALU.
- dest_reg identifies the target register for write-back.
- src1_reg and src2_reg specify the source registers for arithmetic and logical operations.
- immediate provides support for constant operands in future instruction extensions.

These outputs form the interface between the instruction format and the execution logic.

4.6 Result Observation

Simulation results confirm that the decoder correctly extracts all instruction fields according to the defined bit layout. Any change in the instruction input is immediately reflected in the decoder outputs,

validating the correctness of the combinational decoding logic. This behavior ensures reliable instruction interpretation throughout the GPU execution pipeline.

5. Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit (ALU) is a core execution component in processor architectures, including simplified GPU designs. Its primary responsibility is to perform arithmetic and comparison operations on input operands as directed by control signals. In this project, the ALU is designed as a lightweight, combinational execution unit that supports a limited but essential instruction set. This design allows the compute core to evaluate expressions, update registers, and make control-flow decisions during execution.

5.1 ALU Design and Functionality

The ALU supports a small set of fundamental operations required by the compute core, including addition, subtraction, multiplication, and comparison. The specific operation executed is determined by a 4-bit opcode generated by the decoder. Two input operands are provided by the register file, and the ALU produces either a numerical result or a comparison flag depending on the instruction type.

5.2 ALU Implementation

```
// alu.v
`include "definitions.vh"

module alu (
    input [3:0] opcode,
    input [`DATA_WIDTH-1:0] operand_a,
    input [`DATA_WIDTH-1:0] operand_b,
    output reg [`DATA_WIDTH-1:0] result,
    output reg cmp_flag
);
    always @(*) begin
        cmp_flag = 0;
        result = 0;
        case (opcode)
            `OP_ADD: result = operand_a + operand_b;
            `OP_SUB: result = operand_a - operand_b;
            `OP_MUL: result = operand_a * operand_b;
            `OP_CMP: begin
                if (operand_a < operand_b)
                    cmp_flag = 1;
                else
                    cmp_flag = 0;
            end
            default: result = 0;
        endcase
    end
endmodule
```

5.3 Explanation of the ALU Code

The ALU is implemented as a purely combinational module using an always @(*) block, ensuring that outputs update immediately in response to changes in inputs. Both result and cmp_flag are initialized to zero at the beginning of the block to prevent unintended latch inference.

For arithmetic instructions (OP_ADD, OP_SUB, and OP_MUL), the ALU directly computes the result using the two input operands. The comparison instruction (OP_CMP) does not produce a numerical result; instead, it evaluates whether operand_a is less than operand_b and sets the cmp_flag accordingly. Any unsupported or undefined opcode is safely handled by the default case, which forces the output result to zero.

5.4 Result Observation

During simulation, the ALU output reflects the correct arithmetic or comparison outcome for each executed instruction. The computed results are forwarded to the register file for write-back, while the comparison flag is used by control instructions such as conditional branches. This confirms that the ALU operates correctly as the execution engine of the compute core.

6. Compute Core Integration

6.1 Concept of the Compute Core

In this project, a compute core is a simplified processing unit that executes basic instructions such as arithmetic and comparison operations. It represents the smallest functional execution block of the simple GPU model, focusing on clarity and educational purpose rather than performance optimization. The compute core combines instruction control and computation to demonstrate how a GPU processes instructions at a fundamental level.

6.2 Compute Core Implementation in Verilog

```
`include "definitions.vh"

module compute_core (
    input clk,
    input reset,
    output reg halt
);
    reg [`DATA_WIDTH-1:0] register_file [0:`NUM_THREADS-1][0:`REG_COUNT-1];
    reg [3:0] pc [0:`NUM_THREADS-1]; // 4-bit PC for 16 instruction memory entries
    reg [`NUM_THREADS-1:0] active_threads;
    reg [`DATA_WIDTH-1:0] data_mem [0:15];

    // Thread Control
    wire [`THREAD_ID_WIDTH-1:0] scheduled_thread;

    // Instruction Pipeline
    wire [`DATA_WIDTH-1:0] instruction;
    wire [3:0] opcode;
    wire [3:0] dest_reg;
    wire [3:0] src1_reg;
    wire [3:0] src2_reg;
    wire [7:0] immediate;

    // ALU signals
    reg [`DATA_WIDTH-1:0] alu_operand_a;
    reg [`DATA_WIDTH-1:0] alu_operand_b;
    wire [`DATA_WIDTH-1:0] alu_result;
    wire alu_cmp_flag;

    // Per-thread compare flags
    reg [`NUM_THREADS-1:0] cmp_flags;

    integer i, current_thread;
    reg [`NUM_THREADS-1:0] next_active_threads;

    // Instantiate Scheduler
    scheduler scheduler_inst (
        .clk(clk),
        .reset(reset),
        .active_threads(active_threads),
        .scheduled_thread(scheduled_thread)
```

```

);

// Instantiate Fetcher
fetcher fetcher_inst (
    .pc_in(pc[scheduled_thread]),
    .instruction(instruction)
);

// Instantiate Decoder
decoder decoder_inst (
    .instruction(instruction),
    .opcode(opcode),
    .dest_reg(dest_reg),
    .src1_reg(src1_reg),
    .src2_reg(src2_reg),
    .immediate(immediate)
);

// Instantiate ALU
alu alu_inst (
    .opcode(opcode),
    .operand_a(alu_operand_a),
    .operand_b(alu_operand_b),
    .result(alu_result),
    .cmp_flag(alu_cmp_flag)
);

// Initialize data memory
initial begin
    $readmemh("src/data_memory.mem", data_mem);
    $display("Data Memory Initialized:");
    for (i = 0; i < 16; i = i + 1) begin
        $display("data_mem[%0d] = %h", i, data_mem[i]);
    end
end

// Core logic
always @(posedge clk or posedge reset) begin
    if (reset) begin
        for (i = 0; i < `NUM_THREADS; i = i + 1) begin
            pc[i] <= 0;
            active_threads[i] <= 1;
            cmp_flags[i] <= 0;
        end
    end
end

```

```

        for (current_thread = 0; current_thread < `REG_COUNT; current_thread = current_thread + 1)
            register_file[i][current_thread] <= 0;
        end
        register_file[i][15] <= i;
    end
    halt <= 0;
    $display("DUT Reset at time %0t", $time);
end

else if (!halt) begin
    next_active_threads = active_threads;
    current_thread = scheduled_thread;

    if (active_threads[current_thread]) begin
        // Prepare ALU inputs based on instruction type
        case (opcode)
            `OP_ADD, `OP_SUB, `OP_MUL, `OP_CMP: begin
                alu_operand_a = register_file[current_thread][src1_reg];
                alu_operand_b = register_file[current_thread][src2_reg];
            end
            `OP_ADDI, `OP_SUBI: begin
                alu_operand_a = register_file[current_thread][dest_reg];
                alu_operand_b = {{8{immediate[7]}}, immediate}; // Sign extend
            end
            default: begin
                alu_operand_a = 0;
                alu_operand_b = 0;
            end
        endcase

        // Execute instruction
        case (opcode)
            `OP_ADD: begin
                register_file[current_thread][dest_reg] <= alu_result;
                pc[current_thread] <= pc[current_thread] + 1;
            end

            `OP_SUB: begin
                register_file[current_thread][dest_reg] <= alu_result;
                pc[current_thread] <= pc[current_thread] + 1;
            end

            `OP_MUL: begin
                register_file[current_thread][dest_reg] <= alu_result;

```

```

        pc[current_thread] <= pc[current_thread] + 1;
end

`OP_CMP: begin
    cmp_flags[current_thread] <= alu_cmp_flag;
    pc[current_thread] <= pc[current_thread] + 1;
end

`OP_ADDI: begin
    register_file[current_thread][dest_reg] <= alu_result;
    pc[current_thread] <= pc[current_thread] + 1;
end

`OP_SUBI: begin
    register_file[current_thread][dest_reg] <= alu_result;
    pc[current_thread] <= pc[current_thread] + 1;
end

`OP_JMP: begin
    pc[current_thread] <= immediate[3:0]; // Only 4 bits
end

`OP_JLT: begin
    if (cmp_flags[current_thread] == 1) begin
        pc[current_thread] <= immediate[3:0]; // Only 4 bits
    end else begin
        pc[current_thread] <= pc[current_thread] + 1;
    end
end

`OP_LDR: begin
    register_file[current_thread][dest_reg] <= data_mem[immediate[3:0]];
    pc[current_thread] <= pc[current_thread] + 1;
end

`OP_STR: begin
    data_mem[immediate[3:0]] <= register_file[current_thread][dest_reg];
    pc[current_thread] <= pc[current_thread] + 1;
end

`OP_HALT: begin
    next_active_threads[current_thread] = 0;
    $display("Thread %0d executing HALT. Halting.", current_thread);

```

```

        end

        default: begin
            $display("Thread %0d encountered undefined opcode %b. No operation performed");
            current_thread, opcode);
            pc[current_thread] <= pc[current_thread] + 1;
        end
    endcase

    // Logging
    // Logging - different format for R-Type vs I-Type
    $display("Time=%0t | Thread=%0d | PC=%0d | Instruction=%h | Opcode=%b | dest=R%0d | src1=R%0d | src2=R%0d | immediate=%0d",
            $time, current_thread, pc[current_thread], instruction, opcode,
            dest_reg, src1_reg, src2_reg, immediate);
end

active_threads <= next_active_threads;

// Check if all threads have halted
if (next_active_threads == 0) begin
    halt <= 1;
    $display("All threads have halted at time %0t", $time);
end
end
end

// Dump waveforms
initial begin
    $dumpfile("simulation.vcd");
    $dumpvars(0, compute_core);
end

endmodule

```

6.4 Explanation of the Code

- The reset logic initializes the compute core by setting all program counters to zero, clearing register files and comparison flags, and marking all threads as active.
- The scheduler logic selects one active thread index at each clock cycle, ensuring that only runnable threads are issued for execution.
- The instruction fetch block uses the selected thread's program counter to read the corresponding instruction from instruction memory.

- The decoder block extracts opcode and operand fields from the fetched instruction and generates control signals for the ALU, memory access, and control flow logic.
- The ALU execution block performs arithmetic or logical operations based on the decoded opcode and produces computation results and comparison outcomes.
- The register write-back block updates the selected thread's register file with ALU results or loaded memory data.
- The memory access block handles load and store instructions by reading from or writing to shared data memory using computed addresses.
- The control flow block updates the program counter for each thread, either incrementing it normally or modifying it according to branch conditions.
- The halt handling logic marks a thread as inactive when a halt instruction is executed, preventing it from being scheduled again.
- The completion detection logic checks whether all threads are inactive and asserts a done signal to indicate that the compute core has finished execution.

6.5 Output Observation and Execution Result

The output of the compute core provides visibility into the execution state of each thread during simulation. It allows observation of program counter progression, indicating how instructions are fetched and executed over time. The register file outputs show the results of arithmetic and logical operations produced by the ALU. Memory-related outputs reflect the correctness of load and store instructions by exposing accessed addresses and data values. Thread activity signals indicate whether each thread is still active or has reached a halt condition. Finally, the completion signal confirms that all threads have finished execution, demonstrating that the compute core successfully processes a simple parallel workload.

II. Testbench Documentation

1. Verification Methodology

The verification methodology used in this project focuses on module-level functional verification through simulation-based testbenches. Each hardware module is verified independently to ensure correct functionality before system integration.

A directed testing approach is applied, where predefined input patterns are manually provided to the design under test (DUT). This method allows precise control over test scenarios and makes it easier to validate expected outputs for each operation.

All testbenches are written in Verilog and use standard simulation constructs such as:

- *displayand*monitor for observing signal behavior,
- clock and reset generation for sequential modules,
- timeout mechanisms to detect deadlock or incorrect halt conditions.

Verification correctness is evaluated by:

- Comparing output signals with expected results,
- Observing control signals such as halt and scheduling outputs,
- Monitoring internal states (e.g., program counters, registers, memory) when necessary.

This methodology ensures that each module behaves correctly in isolation, providing a reliable foundation for higher-level system verification.

2. Module-Level Testbenches

2.1 Scheduler Testbench

2.1.1 Purpose of the Scheduler Testbench

The Scheduler Testbench is designed to verify the correct operation of the thread scheduling logic. Its main objective is to ensure that the scheduler selects a valid thread based on the current set of active threads and responds correctly to clock and reset signals. The testbench aims to confirm correct scheduling behavior under different thread activity conditions.

2.1.2 Code Analysis

The testbench generates a periodic clock signal with a fixed time period and applies an initial reset to initialize the scheduler state. After the reset is released, various combinations of the `active_threads` input are applied to simulate different scheduling scenarios, including cases where some threads are inactive and cases where all threads are active.

The output signal `scheduled_thread` is monitored at every rising edge of the clock. This allows verification that the scheduler always selects an active thread and that the scheduling behavior remains stable across clock cycles. By testing multiple active thread patterns and reapplying reset during simulation, the testbench verifies both normal operation and reset recovery behavior of the scheduler.

```

module scheduler_tb;

    // Parameters
    parameter NUM_THREADS = 4; // Giả sử có 4 threads
    parameter THREAD_ID_WIDTH = 3; // Sửa chiều rộng ID thread thành 3 bit

    // Inputs
    reg clk;
    reg reset;
    reg [NUM_THREADS-1:0] active_threads; // Tín hiệu xác định các thread còn hoạt động

    // Outputs
    wire [THREAD_ID_WIDTH-1:0] scheduled_thread; // Thread được lập lịch

    // Instantiate the scheduler module
    scheduler #(
        .NUM_THREADS(NUM_THREADS)
    ) uut (
        .clk(clk),
        .reset(reset),
        .active_threads(active_threads),
        .scheduled_thread(scheduled_thread)
    );

    // Clock generation
    always begin
        #5 clk = ~clk; // Chu kỳ đồng hồ 10 đơn vị thời gian
    end

    // Testbench stimulus
    initial begin
        // Initialize signals
        clk = 0;
        reset = 1;
        active_threads = 4'b1111; // Tất cả các thread đều hoạt động

        // Display header
        $display("Time\tReset\tActive Threads\tScheduled Thread");

        // Apply reset and release it after a while
        #10 reset = 0; // Release reset tại thời điểm t = 10

        // Chạy thử với các active_threads khác nhau
    end

```



```

#10 active_threads = 4'b1110; // Thread 3 không hoạt động
#10 active_threads = 4'b1101; // Thread 2 không hoạt động
#10 active_threads = 4'b1011; // Thread 1 không hoạt động
#10 active_threads = 4'b1000; // Chỉ có thread 0 hoạt động

// Chạy với tất cả các thread đều hoạt động
#10 active_threads = 4'b1111;

// Test with reset again
#10 reset = 1; // Thực hiện reset
#10 reset = 0; // Thả reset

// Kết thúc mô phỏng sau khi test đủ các tình huống
#50 $finish;
end

// Monitor output
always @(posedge clk) begin
    $display("%0t\t%0b\t%0b\t%0d", $time, reset, active_threads, scheduled_thread);
end

endmodule

```

output:

Time	Reset	Active Threads	Scheduled Thread
5	1	1111	0
15	0	1111	0
25	0	1110	0
35	0	1101	1
45	0	1011	2
55	0	1000	3
65	0	1111	3
75	1	1111	0
85	0	1111	0
95	0	1111	0
105	0	1111	1
115	0	1111	2
125	0	1111	3

testbench/module/scheduler_tb.v:57: \$finish called at 130 (1s)

2.2 Fetcher Testbench

2.2.1 Purpose of the Fetcher Testbench

The Fetcher Testbench is used to verify the correct instruction fetching behavior based on the program counter (PC) input. Its main purpose is to ensure that the fetcher outputs the correct instruction corresponding to each PC value and correctly interfaces with the instruction memory.

2.2.2 Code Analysis

In the testbench, the program counter input is driven with a sequence of increasing values to simulate sequential instruction execution. Although a 16-bit PC signal is used in the testbench, only the lower bits are connected to the fetcher module, which matches the actual design interface and verifies correct PC width handling.

For each applied PC value, the output instruction is monitored and displayed using `$display`. This enables direct observation of the mapping between PC addresses and fetched instructions during simulation. By sweeping through all valid PC values within the supported range, the testbench confirms that the fetcher accesses instruction memory correctly and produces stable, deterministic outputs.

```

// testbench/module/fetcher_tb.v

module fetcher_tb;
    // Parameters
    parameter ADDR_WIDTH = 16;
    parameter DATA_WIDTH = 16;

    // Inputs
    reg [ADDR_WIDTH-1:0] pc_in;    // Testbench dùng 16-bit

    // Outputs
    wire [DATA_WIDTH-1:0] instruction;

    // Instantiate - CHỈ kết nối ports có thật
    fetcher uut (
        .pc_in(pc_in[3:0]),        // Chỉ truyền 4 bit thấp
        .instruction(instruction)
        // KHÔNG có .instr_mem()!
    );

    // Testbench stimulus
    initial begin
        // Display header
        $display("Time\tPC\tInstruction");

        // Apply test cases
        #10 pc_in = 0;
        #10 pc_in = 1;
        #10 pc_in = 2;
        #10 pc_in = 3;
        #10 pc_in = 4;
        #10 pc_in = 5;
        #10 pc_in = 6;
        #10 pc_in = 7;
        #10 pc_in = 8;
        #10 pc_in = 9;
        #10 pc_in = 10;
        #10 pc_in = 11;
        #10 pc_in = 12;
        #10 pc_in = 13;
        #10 pc_in = 14;
        #10 pc_in = 15;
    end
endmodule

```

```

        // End simulation
        #10 $finish;
    end

    // Monitor
    always @(pc_in) begin
        $display("%0t\t%h\t%h", $time, pc_in[3:0], instruction);
    end

endmodule

```

output:

```

Instruction Memory Initialized in Fetcher:
instr_mem[0] = 6000
instr_mem[1] = 6101
instr_mem[2] = 0221
instr_mem[3] = 8210
instr_mem[4] = 1330
instr_mem[5] = 3032
instr_mem[6] = 500b
instr_mem[7] = 2443
instr_mem[8] = 7402
instr_mem[9] = f000
instr_mem[10] = 0000
instr_mem[11] = 6501
instr_mem[12] = 0550
instr_mem[13] = 7503
instr_mem[14] = f000
instr_mem[15] = 0000

```

Time	PC	Instruction
10	0	6000
20	1	6101
30	2	0221
40	3	8210
50	4	1330
60	5	3032
70	6	500b
80	7	2443
90	8	7402
100	9	f000
110	a	0000
120	b	6501
130	c	0550
140	d	7503
150	e	f000
160	f	0000

```

testbench/module/fetcher_tb.v:45: $finish_ called at 170 (1s)

```

2.3 Decoder Testbench

2.3.1 Purpose of the Decoder Testbench

The Decoder Testbench is designed to verify the correctness of instruction decoding by ensuring that each field within a 16-bit instruction word is properly extracted. Its purpose is to confirm that the decoder correctly generates the opcode, destination register, source register fields, and immediate value for downstream processing.

2.3.2 Code Analysis

In the testbench, several predefined instruction patterns are applied sequentially to the decoder input. These patterns are selected to cover different bit configurations and verify that all decoded output fields respond correctly to changes in the instruction value.

The decoded outputs are continuously monitored and displayed whenever the instruction input changes. This allows verification that the opcode and operand fields are updated consistently and without delay. Through directed test cases and real-time monitoring, the testbench confirms that the decoder accurately interprets instruction formats and produces correct control and data signals for subsequent execution units.

```

// decoder_tb.v

module decoder_tb;

    // Parameters
    parameter DATA_WIDTH = 16; // Chiều rộng của instruction (16 bit)

    // Inputs
    reg [DATA_WIDTH-1:0] instruction;

    // Outputs
    wire [3:0] opcode;
    wire [3:0] dest_reg;
    wire [3:0] src1_reg;
    wire [3:0] src2_reg;
    wire [7:0] immediate;

    // Instantiate the decoder module
    decoder uut (
        .instruction(instruction),
        .opcode(opcode),
        .dest_reg(dest_reg),
        .src1_reg(src1_reg),
        .src2_reg(src2_reg),
        .immediate(immediate)
    );

    // Testbench stimulus
    initial begin
        // Initialize signals
        instruction = 16'b0001_1010_1100_0011; // Một giá trị instruction mẫu

        // Display header
        $display("Time\tInstruction\tOpcode\tDest_Reg\tSrc1_Reg\tSrc2_Reg\tImmediate");

        // Apply different instruction values and monitor output
        #10 instruction = 16'b1001_0101_0110_0111; // Thay đổi giá trị instruction
        #10 instruction = 16'b0110_1111_0001_0010; // Một giá trị instruction khác
        #10 instruction = 16'b1110_0000_1000_1111; // Thêm một giá trị khác

        // End simulation
        #10 $finish;
    end
endmodule

```

```

end

// Monitor outputs during simulation
always @(instruction) begin
    $display("%0t\t%h\t\t%h\t\t%h\t\t%h\t\t%h\t\t%h",
            $time, instruction, opcode, dest_reg, src1_reg, src2_reg, immediate);
end

endmodule

```

output:

Time	Instruction	Opcode	Dest_Reg	Src1_Reg	Src2_Reg	Immediate
0	1ac3	1	a	c	3	c3
10	9567	1	a	c	3	c3
20	6f12	6	f	1	2	12
30	e08f	6	f	1	2	12

testbench/module/decoder_tb.v:43: \$finish_ called at 40 (1s)

2.4 ALU Testbench

2.4.2 Purpose of the ALU Testbench

The ALU Testbench is designed to verify the functional correctness of the Arithmetic Logic Unit (ALU) across multiple operations. Its main purpose is to validate both arithmetic computation results and comparison flags generated by the ALU for different operation types.

2.4.3 Code Analysis

The testbench applies a series of directed test cases, each corresponding to a specific operation code, including addition, subtraction, multiplication, comparison, and immediate addition. For each test case, predefined operand values are provided, and the resulting output is observed.

Simulation outputs are monitored using \$monitor, which displays the operation code, input operands, result, and comparison flag in real time. This allows immediate verification of correctness for each ALU operation. By covering multiple operation types and operand combinations, the testbench confirms that the ALU produces accurate results and correctly asserts control flags for use within the compute core.

```

module alu_tb;

    reg [3:0] opcode;
    reg [`DATA_WIDTH-1:0] operand_a;
    reg [`DATA_WIDTH-1:0] operand_b;

    wire [`DATA_WIDTH-1:0] result;
    wire cmp_flag;

    alu uut (
        .opcode(opcode),
        .operand_a(operand_a),
        .operand_b(operand_b),
        .result(result),
        .cmp_flag(cmp_flag)
    );

    initial begin
        opcode      = 4'b0000;
        operand_a   = 16'h0000;
        operand_b   = 16'h0000;

        $monitor("T=%0t | op=%b | A=%d | B=%d | R=%d | CMP=%b",
            $time, opcode, operand_a, operand_b, result, cmp_flag);

        // ADD: 7 + 5 = 12
        #10 opcode = `OP_ADD;  operand_a = 16'd7;  operand_b = 16'd5;
        #10;

        // SUB: 9 - 4 = 5
        #10 opcode = `OP_SUB;  operand_a = 16'd9;  operand_b = 16'd4;
        #10;

        // MUL: 6 * 3 = 18
        #10 opcode = `OP_MUL;  operand_a = 16'd6;  operand_b = 16'd3;
        #10;

        // CMP: 2 < 8 → cmp_flag = 1
        #10 opcode = `OP_CMP;  operand_a = 16'd2;  operand_b = 16'd8;
        #10;

        // ADDI: 10 + 15 = 25
        #10 opcode = `OP_ADDI; operand_a = 16'd10; operand_b = 16'd15;

```



```

        #10;

        $finish;
    end
endmodule

```

output:

```

T=0 | op=0000 | A= 0 | B= 0 | R= 0 | CMP=0
T=10 | op=0000 | A= 7 | B= 5 | R= 12 | CMP=0
T=30 | op=0001 | A= 9 | B= 4 | R= 5 | CMP=0
T=50 | op=0010 | A= 6 | B= 3 | R= 18 | CMP=0
T=70 | op=0011 | A= 2 | B= 8 | R= 0 | CMP=1
T=90 | op=1000 | A= 10 | B= 15 | R= 0 | CMP=0
testbench/module/alu_tb.v:47: $finish called at 100 (1s)

```

2.5 Compute Core Testbench

2.5.1 Purpose of the Compute Core Testbench

The Compute Core Testbench is designed to verify the overall behavior of the compute core by validating correct execution flow and coordination among internal components. Its purpose is to ensure proper instruction execution, thread control, and correct operation of registers and data memory.

2.5.2 Code Analysis

The testbench generates a periodic clock signal and applies a single reset pulse to initialize the compute core. After the reset is released, the core executes instructions autonomously until the halt signal is asserted.

During simulation, program counter values for all threads are monitored to observe control flow progression. A timeout mechanism is included to detect potential deadlock conditions if the halt signal is not asserted within the expected time frame.

Once execution completes, the testbench reports final register contents and data memory values for each thread. Performance metrics, including total cycle count and average cycles per thread, are also displayed. Through these mechanisms, the testbench confirms correct execution behavior, proper thread termination, and overall functional integration of the compute core.

```

// compute_core_tb.v
`include "definitions.vh"

module compute_core_tb;

    // Inputs
    reg clk;
    reg reset;

    // Outputs
    wire halt;

    // Instantiate the compute_core module
    compute_core uut (
        .clk(clk),
        .reset(reset),
        .halt(halt)
    );

    // Clock generation: 10 time unit period
    initial clk = 0;
    always #5 clk = ~clk;

    // Testbench stimulus
    initial begin
        $display("=====");
        $display("    Compute Core Testbench Start");
        $display("=====");
        $display("Time\tReset\tHalt\tPC[0]\tPC[1]\tPC[2]\tPC[3]\tPC[4]\tPC[5]\tPC[6]\tPC[7]");

        // Initialize signals
        reset = 1;

        // Apply reset pulse (single time only!)
        #10 reset = 0;

        $display("\n[INFO] Reset released. Starting computation...\n");

        // Wait for halt signal or timeout
        wait(halt == 1);

        $display("\n=====");
        $display("    All Threads Halted - Test Passed!");
    end

```

```

$display("=====");

// Display final register states
$display("\nFinal Register States:");
$display("Thread 0: R0=%h R1=%h R2=%h R3=%h",
        uut.register_file[0][0], uut.register_file[0][1],
        uut.register_file[0][2], uut.register_file[0][3]);
$display("Thread 1: R0=%h R1=%h R2=%h R3=%h",
        uut.register_file[1][0], uut.register_file[1][1],
        uut.register_file[1][2], uut.register_file[1][3]);

// Display final data memory
$display("\nFinal Data Memory:");
$display("mem[0]=%h mem[1]=%h mem[2]=%h mem[3]=%h",
        uut.data_mem[0], uut.data_mem[1], uut.data_mem[2], uut.data_mem[3]);

#50; // Wait a bit after halt
$finish;
end

// Timeout mechanism - in case halt never asserts
initial begin
    #50000; // 50000 time units = 5000 cycles timeout
    $display("\n=====");
    $display("    ERROR: Simulation Timeout!");
    $display("    Halt signal never asserted");
    $display("=====");
    $finish;
end

// Monitor PC values every clock cycle
always @(posedge clk) begin
    if (!reset) begin // Only display when not in reset
        $display("%0t\t%0b\t%0b\t%h\t%h\t%h\t%h\t%h\t%h\t%h",
            $time, reset, halt,
            uut.pc[0], uut.pc[1], uut.pc[2], uut.pc[3],
            uut.pc[4], uut.pc[5], uut.pc[6], uut.pc[7]);
    end
end

// Monitor halt signal
always @(posedge halt) begin
    $display("\n[INFO] Halt signal asserted at time %0t", $time);
end

```

end

// Performance counter

integer cycle_count;

initial cycle_count = 0;

always @(posedge clk) begin

if (!reset && !halt)

cycle_count = cycle_count + 1;

end

// Display performance metrics at end

initial begin

wait(halt == 1);

#10;

\$display("\nPerformance Metrics:");

\$display("Total cycles executed: %0d", cycle_count);

\$display("Average cycles per thread: %0d", cycle_count / `NUM_THREADS);

end

endmodule

output:

Instruction Memory Initialized in Fetcher:

instr_mem[0] = 6000

instr_mem[1] = 6101

instr_mem[2] = 0221

instr_mem[3] = 8210

instr_mem[4] = 1330

instr_mem[5] = 3032

instr_mem[6] = 500b

instr_mem[7] = 2443

instr_mem[8] = 7402

instr_mem[9] = f000

instr_mem[10] = 0000

instr_mem[11] = 6501

instr_mem[12] = 0550

instr_mem[13] = 7503

instr_mem[14] = f000

instr_mem[15] = 0000

Data Memory Initialized:

data_mem[0] = 0003

data_mem[1] = 0005

data_mem[2] = 0000

data_mem[3] = 0000

data_mem[4] = 0000

data_mem[5] = 0000

data_mem[6] = 0000

data_mem[7] = 0000

data_mem[8] = 0000

data_mem[9] = 0000

data_mem[10] = 0000

data_mem[11] = 0000

data_mem[12] = 0000

data_mem[13] = 0000

data_mem[14] = 0000

data_mem[15] = 0000

VCD info: dumpfile simulation.vcd opened for output.

=====

Compute Core Testbench Start

=====

Time Reset Halt PC[0] PC[1] PC[2] PC[3] PC[4] PC[5] PC[6] PC[7]
DUT Reset at time 0
DUT Reset at time 5

[INF0] Reset released. Starting computation...

15	0	0	0	0	0	0	0	0	0	0	0					
Time=15		Thread=0		PC=0		Instruction=6000		Opcode=0110		dest=R0		src1=R0		src2=R0		imm= 0
25	0	0	1	0	0	0	0	0	0	0	0					
Time=25		Thread=0		PC=1		Instruction=6101		Opcode=0110		dest=R1		src1=R0		src2=R1		imm= 1
35	0	0	2	0	0	0	0	0	0	0	0					
Time=35		Thread=1		PC=0		Instruction=6000		Opcode=0110		dest=R0		src1=R0		src2=R0		imm= 0
45	0	0	2	1	0	0	0	0	0	0	0					
Time=45		Thread=2		PC=0		Instruction=6000		Opcode=0110		dest=R0		src1=R0		src2=R0		imm= 0
55	0	0	2	1	1	0	0	0	0	0	0					
Time=55		Thread=3		PC=0		Instruction=6000		Opcode=0110		dest=R0		src1=R0		src2=R0		imm= 0
65	0	0	2	1	1	1	0	0	0	0	0					
Time=65		Thread=4		PC=0		Instruction=6000		Opcode=0110		dest=R0		src1=R0		src2=R0		imm= 0
75	0	0	2	1	1	1	1	0	0	0	0					
Time=75		Thread=5		PC=0		Instruction=6000		Opcode=0110		dest=R0		src1=R0		src2=R0		imm= 0
85	0	0	2	1	1	1	1	1	0	0	0					
Time=85		Thread=6		PC=0		Instruction=6000		Opcode=0110		dest=R0		src1=R0		src2=R0		imm= 0
95	0	0	2	1	1	1	1	1	1	0	0					
Time=95		Thread=7		PC=0		Instruction=6000		Opcode=0110		dest=R0		src1=R0		src2=R0		imm= 0
105	0	0	2	1	1	1	1	1	1	1	1					
Time=105		Thread=0		PC=2		Instruction=0221		Opcode=0000		dest=R2		src1=R2		src2=R1		imm= 33
115	0	0	3	1	1	1	1	1	1	1	1					
Time=115		Thread=1		PC=1		Instruction=6101		Opcode=0110		dest=R1		src1=R0		src2=R1		imm= 1
125	0	0	3	2	1	1	1	1	1	1	1					
Time=125		Thread=2		PC=1		Instruction=6101		Opcode=0110		dest=R1		src1=R0		src2=R1		imm= 1
135	0	0	3	2	2	1	1	1	1	1	1					
Time=135		Thread=3		PC=1		Instruction=6101		Opcode=0110		dest=R1		src1=R0		src2=R1		imm= 1
145	0	0	3	2	2	2	1	1	1	1	1					
Time=145		Thread=4		PC=1		Instruction=6101		Opcode=0110		dest=R1		src1=R0		src2=R1		imm= 1
155	0	0	3	2	2	2	2	1	1	1	1					
Time=155		Thread=5		PC=1		Instruction=6101		Opcode=0110		dest=R1		src1=R0		src2=R1		imm= 1
165	0	0	3	2	2	2	2	2	1	1	1					
Time=165		Thread=6		PC=1		Instruction=6101		Opcode=0110		dest=R1		src1=R0		src2=R1		imm= 1
175	0	0	3	2	2	2	2	2	2	1	1					
Time=175		Thread=7		PC=1		Instruction=6101		Opcode=0110		dest=R1		src1=R0		src2=R1		imm= 1
185	0	0	3	2	2	2	2	2	2	2	2					
Time=185		Thread=0		PC=3		Instruction=8210		Opcode=1000		dest=R2		src1=R1		src2=R0		imm= 16
195	0	0	4	2	2	2	2	2	2	2	2					
Time=195		Thread=1		PC=2		Instruction=0221		Opcode=0000		dest=R2		src1=R2		src2=R1		imm= 33
205	0	0	4	3	2	2	2	2	2	2	2					
Time=205		Thread=2		PC=2		Instruction=0221		Opcode=0000		dest=R2		src1=R2		src2=R1		imm= 33
215	0	0	4	3	3	2	2	2	2	2	2					
Time=215		Thread=3		PC=2		Instruction=0221		Opcode=0000		dest=R2		src1=R2		src2=R1		imm= 33
225	0	0	4	3	3	3	2	2	2	2	2					
Time=225		Thread=4		PC=2		Instruction=0221		Opcode=0000		dest=R2		src1=R2		src2=R1		imm= 33
235	0	0	4	3	3	3	3	2	2	2	2					
Time=235		Thread=5		PC=2		Instruction=0221		Opcode=0000		dest=R2		src1=R2		src2=R1		imm= 33
245	0	0	4	3	3	3	3	3	2	2	2					
Time=245		Thread=6		PC=2		Instruction=0221		Opcode=0000		dest=R2		src1=R2		src2=R1		imm= 33
255	0	0	4	3	3	3	3	3	3	2	2					
Time=255		Thread=7		PC=2		Instruction=0221		Opcode=0000		dest=R2		src1=R2		src2=R1		imm= 33
265	0	0	4	3	3	3	3	3	3	3	3					
Time=265		Thread=0		PC=4		Instruction=1330		Opcode=0001		dest=R3		src1=R3		src2=R0		imm= 48
275	0	0	5	3	3	3	3	3	3	3	3					
Time=275		Thread=1		PC=3		Instruction=8210		Opcode=1000		dest=R2		src1=R1		src2=R0		imm= 16
285	0	0	5	4	3	3	3	3	3	3	3					
Time=285		Thread=2		PC=3		Instruction=8210		Opcode=1000		dest=R2		src1=R1		src2=R0		imm= 16
295	0	0	5	4	4	3	3	3	3	3	3					
Time=295		Thread=3		PC=3		Instruction=8210		Opcode=1000		dest=R2		src1=R1		src2=R0		imm= 16
305	0	0	5	4	4	4	3	3	3	3	3					
Time=305		Thread=4		PC=3		Instruction=8210		Opcode=1000		dest=R2		src1=R1		src2=R0		imm= 16
315	0	0	5	4	4	4	4	3	3	3	3					
Time=315		Thread=5		PC=3		Instruction=8210		Opcode=1000		dest=R2		src1=R1		src2=R0		imm= 16
325	0	0	5	4	4	4	4	4	3	3	3					
Time=325		Thread=6		PC=3		Instruction=8210		Opcode=1000		dest=R2		src1=R1		src2=R0		imm= 16
335	0	0	5	4	4	4	4	4	4	3	3					
Time=335		Thread=7		PC=3		Instruction=8210		Opcode=1000		dest=R2		src1=R1		src2=R0		imm= 16
345	0	0	5	4	4	4	4	4	4	4	4					
Time=345		Thread=0		PC=5		Instruction=3032		Opcode=0011		dest=R0		src1=R3		src2=R2		imm= 50
355	0	0	6	4	4	4	4	4	4	4	4					
Time=355		Thread=1		PC=4		Instruction=1330		Opcode=0001		dest=R3		src1=R3		src2=R0		imm= 48
365	0	0	6	5	4	4	4	4	4	4	4					
Time=365		Thread=2		PC=4		Instruction=1330		Opcode=0001		dest=R3		src1=R3		src2=R0		imm= 48
375	0	0	6	5	5	4	4	4	4	4	4					
Time=375		Thread=3		PC=4		Instruction=1330		Opcode=0001		dest=R3		src1=R3		src2=R0		imm= 48
385	0	0	6	5	5	5	4	4	4	4	4					
Time=385		Thread=4		PC=4		Instruction=1330		Opcode=0001		dest=R3		src1=R3		src2=R0		imm= 48
395	0	0	6	5	5	5	5	4	4	4	4					
Time=395		Thread=5		PC=4		Instruction=1330		Opcode=0001		dest=R3		src1=R3		src2=R0		imm= 48
405	0	0	6	5	5	5	5	5	4	4	4					
Time=405		Thread=6		PC=4		Instruction=1330		Opcode=0001		dest=R3		src1=R3		src2=R0		imm= 48
415	0	0	6	5	5	5	5	5	5	4	4					
Time=415		Thread=7		PC=4		Instruction=1330		Opcode=0001		dest=R3		src1=R3		src2=R0		imm= 48
425	0	0	6	5	5	5	5	5	5	5	5					
Time=425		Thread=0		PC=6		Instruction=500b		Opcode=0101		dest=R0		src1=R0		src2=R11		imm= 11

435	0	0	b	5	5	5	5	5	5	5	5					
Time=435		Thread=1		PC=5		Instruction=3032		Opcode=0011		dest=R0		src1=R3		src2=R2		imm= 50
445	0	0	b	6	5	5	5	5	5	5	5					
Time=445		Thread=2		PC=5		Instruction=3032		Opcode=0011		dest=R0		src1=R3		src2=R2		imm= 50
455	0	0	b	6	6	5	5	5	5	5	5					
Time=455		Thread=3		PC=5		Instruction=3032		Opcode=0011		dest=R0		src1=R3		src2=R2		imm= 50
465	0	0	b	6	6	6	5	5	5	5	5					
Time=465		Thread=4		PC=5		Instruction=3032		Opcode=0011		dest=R0		src1=R3		src2=R2		imm= 50
475	0	0	b	6	6	6	6	5	5	5	5					
Time=475		Thread=5		PC=5		Instruction=3032		Opcode=0011		dest=R0		src1=R3		src2=R2		imm= 50
485	0	0	b	6	6	6	6	6	5	5	5					
Time=485		Thread=6		PC=5		Instruction=3032		Opcode=0011		dest=R0		src1=R3		src2=R2		imm= 50
495	0	0	b	6	6	6	6	6	6	5	5					
Time=495		Thread=7		PC=5		Instruction=3032		Opcode=0011		dest=R0		src1=R3		src2=R2		imm= 50
505	0	0	b	6	6	6	6	6	6	6	6					
Time=505		Thread=0		PC=11		Instruction=6501		Opcode=0110		dest=R5		src1=R0		src2=R1		imm= 1
515	0	0	c	6	6	6	6	6	6	6	6					
Time=515		Thread=1		PC=6		Instruction=500b		Opcode=0101		dest=R0		src1=R0		src2=R11		imm= 11
525	0	0	c	7	6	6	6	6	6	6	6					
Time=525		Thread=2		PC=6		Instruction=500b		Opcode=0101		dest=R0		src1=R0		src2=R11		imm= 11
535	0	0	c	7	7	6	6	6	6	6	6					
Time=535		Thread=3		PC=6		Instruction=500b		Opcode=0101		dest=R0		src1=R0		src2=R11		imm= 11
545	0	0	c	7	7	7	6	6	6	6	6					
Time=545		Thread=4		PC=6		Instruction=500b		Opcode=0101		dest=R0		src1=R0		src2=R11		imm= 11
555	0	0	c	7	7	7	7	6	6	6	6					
Time=555		Thread=5		PC=6		Instruction=500b		Opcode=0101		dest=R0		src1=R0		src2=R11		imm= 11
565	0	0	c	7	7	7	7	7	6	6	6					
Time=565		Thread=6		PC=6		Instruction=500b		Opcode=0101		dest=R0		src1=R0		src2=R11		imm= 11
575	0	0	c	7	7	7	7	7	7	6	6					
Time=575		Thread=7		PC=6		Instruction=500b		Opcode=0101		dest=R0		src1=R0		src2=R11		imm= 11
585	0	0	c	7	7	7	7	7	7	7	7					
Time=585		Thread=0		PC=12		Instruction=0550		Opcode=0000		dest=R5		src1=R5		src2=R0		imm= 80
595	0	0	d	7	7	7	7	7	7	7	7					
Time=595		Thread=1		PC=7		Instruction=2443		Opcode=0010		dest=R4		src1=R4		src2=R3		imm= 67
605	0	0	d	8	7	7	7	7	7	7	7					
Time=605		Thread=2		PC=7		Instruction=2443		Opcode=0010		dest=R4		src1=R4		src2=R3		imm= 67
615	0	0	d	8	8	7	7	7	7	7	7					
Time=615		Thread=3		PC=7		Instruction=2443		Opcode=0010		dest=R4		src1=R4		src2=R3		imm= 67
625	0	0	d	8	8	8	7	7	7	7	7					
Time=625		Thread=4		PC=7		Instruction=2443		Opcode=0010		dest=R4		src1=R4		src2=R3		imm= 67
635	0	0	d	8	8	8	8	7	7	7	7					
Time=635		Thread=5		PC=7		Instruction=2443		Opcode=0010		dest=R4		src1=R4		src2=R3		imm= 67
645	0	0	d	8	8	8	8	8	7	7	7					
Time=645		Thread=6		PC=7		Instruction=2443		Opcode=0010		dest=R4		src1=R4		src2=R3		imm= 67
655	0	0	d	8	8	8	8	8	8	7	7					
Time=655		Thread=7		PC=7		Instruction=2443		Opcode=0010		dest=R4		src1=R4		src2=R3		imm= 67
665	0	0	d	8	8	8	8	8	8	8	8					
Time=665		Thread=0		PC=13		Instruction=7503		Opcode=0111		dest=R5		src1=R0		src2=R3		imm= 3
675	0	0	e	8	8	8	8	8	8	8	8					
Time=675		Thread=1		PC=8		Instruction=7402		Opcode=0111		dest=R4		src1=R0		src2=R2		imm= 2
685	0	0	e	9	8	8	8	8	8	8	8					
Time=685		Thread=2		PC=8		Instruction=7402		Opcode=0111		dest=R4		src1=R0		src2=R2		imm= 2
695	0	0	e	9	9	8	8	8	8	8	8					
Time=695		Thread=3		PC=8		Instruction=7402		Opcode=0111		dest=R4		src1=R0		src2=R2		imm= 2
705	0	0	e	9	9	9	8	8	8	8	8					
Time=705		Thread=4		PC=8		Instruction=7402		Opcode=0111		dest=R4		src1=R0		src2=R2		imm= 2
715	0	0	e	9	9	9	9	8	8	8	8					
Time=715		Thread=5		PC=8		Instruction=7402		Opcode=0111		dest=R4		src1=R0		src2=R2		imm= 2
725	0	0	e	9	9	9	9	9	8	8	8					
Time=725		Thread=6		PC=8		Instruction=7402		Opcode=0111		dest=R4		src1=R0		src2=R2		imm= 2
735	0	0	e	9	9	9	9	9	9	8	8					
Time=735		Thread=7		PC=8		Instruction=7402		Opcode=0111		dest=R4		src1=R0		src2=R2		imm= 2
745	0	0	e	9	9	9	9	9	9	9	9					
Thread 0 executing HALT. Halting.																
Time=745		Thread=0		PC=14		Instruction=f000		Opcode=1111		dest=R0		src1=R0		src2=R0		imm= 0
755	0	0	e	9	9	9	9	9	9	9	9					
Thread 1 executing HALT. Halting.																
Time=755		Thread=1		PC=9		Instruction=f000		Opcode=1111		dest=R0		src1=R0		src2=R0		imm= 0
765	0	0	e	9	9	9	9	9	9	9	9					
Thread 2 executing HALT. Halting.																
Time=765		Thread=2		PC=9		Instruction=f000		Opcode=1111		dest=R0		src1=R0		src2=R0		imm= 0
775	0	0	e	9	9	9	9	9	9	9	9					
Thread 3 executing HALT. Halting.																
Time=775		Thread=3		PC=9		Instruction=f000		Opcode=1111		dest=R0		src1=R0		src2=R0		imm= 0
785	0	0	e	9	9	9	9	9	9	9	9					
Thread 4 executing HALT. Halting.																
Time=785		Thread=4		PC=9		Instruction=f000		Opcode=1111		dest=R0		src1=R0		src2=R0		imm= 0
795	0	0	e	9	9	9	9	9	9	9	9					
Thread 5 executing HALT. Halting.																
Time=795		Thread=5		PC=9		Instruction=f000		Opcode=1111		dest=R0		src1=R0		src2=R0		imm= 0
805	0	0	e	9	9	9	9	9	9	9	9					
Thread 6 executing HALT. Halting.																
Time=805		Thread=6		PC=9		Instruction=f000		Opcode=1111		dest=R0		src1=R0		src2=R0		imm= 0
815	0	0	e	9	9	9	9	9	9	9	9					
Thread 7 executing HALT. Halting.																
Time=815		Thread=7		PC=9		Instruction=f000		Opcode=1111		dest=R0		src1=R0		src2=R0		imm= 0

```

=====
All Threads Halted - Test Passed!
=====

Final Register States:
Thread 0: R0=0003 R1=0005 R2=0000 R3=ffff
Thread 1: R0=0003 R1=0005 R2=0000 R3=ffff

Final Data Memory:
mem[0]=0003 mem[1]=0005 mem[2]=0000 mem[3]=0000

[INFO] Halt signal asserted at time 815

Performance Metrics:
Total cycles executed: 81
Average cycles per thread: 10
825  0      1      e      9      9      9      9      9      9      9
835  0      1      e      9      9      9      9      9      9      9
845  0      1      e      9      9      9      9      9      9      9
855  0      1      e      9      9      9      9      9      9      9
testbench/module/compute_core_tb.v:61: $finish called at 865 (1s)
865  0      1      e      9      9      9      9      9      9      9

```

3. GPU Simulation Using Cocotb

3.1 The role of Compute Core

As mentioned, The Compute Core represents the primary execution engine of the GPU, responsible for instruction processing, multi-threaded scheduling, arithmetic operations, memory access, and control flow. As all major GPU functionalities converge at the Compute Core, its successful verification provides strong evidence of the correctness of the overall GPU operation. Consequently, validating the Compute Core through simulation effectively demonstrates the functional behavior of the GPU architecture as a whole.

3.2 Simulation Framework Overview

To verify the Compute Core and, by extension, the GPU's operational behavior, the design is simulated using Cocotb, a Python-based verification framework, together with the Icarus Verilog simulator. This environment enables high-level, programmable test scenarios while preserving cycle-accurate interaction with the hardware description.

The simulation framework integrates the GPU hardware modules, memory initialization files, and a Cocotb-based testbench to form a complete end-to-end verification flow.

3.3 Simulation Architecture and Setup

The hardware design includes the Compute Core and its supporting modules, such as the scheduler, fetcher, decoder, and arithmetic logic unit (ALU). An external Python-based assembler translates assembly programs into machine-code memory images, which are loaded into the instruction memory prior to simulation. Data memory is initialized from a hexadecimal file and shared across all executing threads.

The hardware design is compiled using Icarus Verilog, while Cocotb orchestrates the simulation by generating the clock signal, applying the reset sequence, and supervising program execution. This

integrated environment enables end-to-end verification of the compute core and demonstrates the fundamental operational behavior of the GPU.

To realize this simulation flow, three main components are used: the Python assembler, the assembly-level test program, and the Cocotb testbench. The assembler generates the instruction memory image, the test program defines the workload executed by the GPU, and the Cocotb testbench controls and observes the simulation.

3.3.1 Python Assembler

The Python assembler converts human-readable assembly code into machine instructions. This allows flexible development and rapid testing of GPU programs. The assembler supports arithmetic, comparison, memory access, control flow, and halt instructions. Registers are encoded using 4-bit identifiers. A two-pass approach is used. The first pass collects labels and constants. The second pass generates the hexadecimal instruction memory file. This file is loaded automatically by the fetcher module during simulation.


```
# Python program to convert assembly code to instructions fitting for the gpu at instruction_mer
```

```
import sys
```

```
import re
```

```
# Define opcode mappings
```

```
opcode_map = {  
    'ADD': '0000',  
    'SUB': '0001',  
    'MUL': '0010',  
    'CMP': '0011',  
    'JMP': '0100',  
    'JLT': '0101', # Jump if Less Than  
    'LDRI': '0110', # Load Immediate  
    'LDR': '0110', # Load from Memory  
    'STR': '0111',  
    'HALT': '1111',  
}
```

```
# Define register mappings (R0 to R15)
```

```
register_map = {f'R{i}': f'{i:04b}' for i in range(16)}
```

```
def assemble_instruction(parts, labels, definitions):
```

```
    if not parts:
```

```
        return None
```

```
    opcode = parts[0].upper()
```

```
    opcode_bin = opcode_map.get(opcode, None)
```

```
    if opcode_bin is None:
```

```
        print(f"Unknown opcode: {opcode}")
```

```
        return None
```

```
    if opcode in ['HALT']:
```

```
        # HALT has no operands
```

```
        return opcode_bin + '0000' + '00000000'
```

```
    elif opcode in ['LDR', 'STR']:
```

```
        # LDR dest_reg, [address]
```

```
        # STR src_reg, [address]
```

```
        if len(parts) != 3:
```

```
            print(f"Incorrect number of operands for {opcode}: {parts}")
```

```
            return None
```

```
        reg = parts[1].rstrip(',').upper()
```

```
        operand = parts[2].strip('[]').upper()
```

```

# Resolve operand (address)
if operand in labels:
    address = labels[operand]
elif operand in definitions:
    address = definitions[operand]
else:
    try:
        if operand.startswith('0x') or operand.startswith('0X'):
            address = int(operand, 16)
        else:
            address = int(operand)
    except ValueError:
        print(f"Invalid operand address: {operand}")
        return None
dest_reg_bin = register_map.get(reg, '0000')
immediate_bin = f'{address & 0xFF:08b}' # 8-bit immediate
return opcode_bin + dest_reg_bin + immediate_bin

elif opcode in ['ADD', 'SUB', 'MUL', 'CMP']:
    # ADD dest_reg, src_reg, immediate
    if len(parts) != 4:
        print(f"Incorrect number of operands for {opcode}: {parts}")
        return None
    dest_reg = parts[1].rstrip(',').upper()
    src_reg = parts[2].rstrip(',').upper()
    immediate = parts[3].upper()
    # Resolve immediate
    if immediate in labels:
        imm_value = labels[immediate]
    elif immediate in definitions:
        imm_value = definitions[immediate]
    else:
        try:
            if immediate.startswith('#'):
                imm_value = int(immediate[1:])
            elif immediate.startswith('0x') or immediate.startswith('0X'):
                imm_value = int(immediate, 16)
            else:
                imm_value = int(immediate)
        except ValueError:
            print(f"Invalid immediate value: {immediate}")
            return None
    dest_reg_bin = register_map.get(dest_reg, '0000')

```

```

src_reg_bin = register_map.get(src_reg, '0000')
immediate_bin = f'{imm_value & 0x0F:04b}' # 4-bit immediate
return opcode_bin + dest_reg_bin + src_reg_bin + immediate_bin

elif opcode in ['JMP', 'JLT']:
    # JMP label
    # JLT label
    if len(parts) != 2:
        print(f"Incorrect number of operands for {opcode}: {parts}")
        return None
    label = parts[1].upper()
    if label in labels:
        address = labels[label]
    elif label in definitions:
        address = definitions[label]
    else:
        try:
            if label.startswith('0x') or label.startswith('0X'):
                address = int(label, 16)
            else:
                address = int(label)
        except ValueError:
            print(f"Invalid jump address: {label}")
            return None
    # For JMP and JLT, use a special register or ignore
    dest_reg_bin = '0000' # Assuming no destination register for jumps
    immediate_bin = f'{address & 0xFF:08b}'
    return opcode_bin + dest_reg_bin + immediate_bin

elif opcode in ['LDRI']:
    # LDRI dest_reg, immediate
    if len(parts) != 3:
        print(f"Incorrect number of operands for {opcode}: {parts}")
        return None
    dest_reg = parts[1].rstrip(',').upper()
    immediate = parts[2].upper()
    # Resolve immediate
    if immediate in labels:
        imm_value = labels[immediate]
    elif immediate in definitions:
        imm_value = definitions[immediate]
    else:
        try:

```

```

        if immediate.startswith('#'):
            imm_value = int(immediate[1:])
        elif immediate.startswith('0x') or immediate.startswith('0X'):
            imm_value = int(immediate, 16)
        else:
            imm_value = int(immediate)
    except ValueError:
        print(f"Invalid immediate value: {immediate}")
        return None

    dest_reg_bin = register_map.get(dest_reg, '0000')
    immediate_bin = f'{imm_value & 0xFF:08b}' # 8-bit immediate
    return opcode_bin + dest_reg_bin + immediate_bin

else:
    print(f"Unhandled opcode: {opcode}")
    return None

def assemble(lines):
    labels = {}
    definitions = {}
    machine_code = []
    address = 0

    # First pass: collect labels and definitions
    for line in lines:
        line = line.strip()
        # Remove comments
        line = re.split(r'#|//', line)[0].strip()
        if not line:
            continue
        # Handle .define
        if line.startswith('.define'):
            parts = line.split()
            if len(parts) != 3:
                print(f"Invalid .define directive: {line}")
                continue
            _, name, value = parts
            try:
                if value.startswith('0x') or value.startswith('0X'):
                    definitions[name.upper()] = int(value, 16)
                else:
                    definitions[name.upper()] = int(value)
            except ValueError:

```

```

        print(f"Invalid .define value: {line}")
        continue
# Handle labels
if line.endswith(':'):
    label = line[:-1].upper()
    labels[label] = address
    continue
# Otherwise, it's an instruction
address += 1

# Second pass: assemble instructions
address = 0
for line in lines:
    original_line = line
    line = line.strip()
    # Remove comments
    line = re.split(r'#|//', line)[0].strip()
    if not line:
        continue
    # Handle .define
    if line.startswith('.define'):
        continue
    # Handle labels
    if line.endswith(':'):
        continue
    # Split instruction into parts
    parts = re.split(r'[, \s]+', line)
    instruction_bin = assemble_instruction(parts, labels, definitions)
    if instruction_bin:
        instruction_hex = f'{int(instruction_bin, 2):04X}'
        machine_code.append(instruction_hex)
        address += 1
return machine_code

def main():
    if len(sys.argv) < 2:
        print("Usage: python assembler.py assembly_code.asm")
        return

    asm_file = sys.argv[1]
    with open(asm_file, 'r') as f:
        lines = f.readlines()

```

```

machine_code = assemble(lines)
with open('instruction_memory.mem', 'w') as f:
    for code in machine_code:
        f.write(code + '\n')

print("Assembly complete. Machine code written to instruction_memory.mem.")

if __name__ == '__main__':
    main()

```

3.3.2 Assembly Test Program

A non-trivial assembly test program was developed to validate the compute core. The program performs an iterative two-dimensional computation resembling a Mandelbrot-style workload. Symbolic constants define iteration limits, thresholds, and initial values. These are resolved by the assembler. Nested loops iterate over X and Y coordinates. An inner loop performs repeated arithmetic and comparison operations. The program uses conditional jumps and memory stores to control execution and write results. Successful completion, indicated by all threads executing HALT, confirms correct functional integration.

```
// A program written in assembly that can be translated into instructions at instruction_memory.
```

```
.define MAX_ITER 16
.define THRESHOLD 1024
.define VALUE_C_REAL 10
.define VALUE_C_IMAG 10
.define X_INIT 0
.define Y_INIT 0
.define Z_REAL_INIT 0
.define Z_IMAG_INIT 0
.define MAX_X 16
.define MAX_Y 16
```

```
LDRI R11, [MAX_ITER]
LDRI R12, [THRESHOLD]
```

```
LDRI R1, [Y_INIT]
```

```
LOOP_Y:
    CMP R1, [MAX_Y]
    JLT LOOP_Y_BODY
    JMP END_PROGRAM
```

```
LOOP_Y_BODY:

    LDRI R0, [X_INIT]
```

```
LOOP_X:
    CMP R0, [MAX_X]
    JLT LOOP_X_BODY
    ADD R1, R1, #1
    JMP LOOP_Y
```

```
LOOP_X_BODY:

    LDRI R4, [VALUE_C_REAL]
    LDRI R5, [VALUE_C_IMAG]

    LDRI R2, [Z_REAL_INIT]
    LDRI R3, [Z_IMAG_INIT]
```

```
LDRI R10, [ITER_INIT]
```

```
LOOP_MANDEL:
```

```
MUL R6, R2, R2
```

```
MUL R7, R3, R3
```

```
MUL R8, R2, R3
```

```
ADD R8, R8, #0
```

```
SUB R2, R6, R7
```

```
ADD R2, R2, R4
```

```
ADD R3, R8, R5
```

```
ADD R9, R6, R7
```

```
CMP R9, R12
```

```
JLT CONTINUE_ITER
```

```
MUL R14, R1, #16
```

```
ADD R14, R14, R0
```

```
STR R10, [R14]
```

```
ADD R0, R0, #1
```

```
JMP LOOP_X
```

```
CONTINUE_ITER:
```

```
ADD R10, R10, #1
```



```
CMP R10, R11
JLT LOOP_MANDEL
```

```
MUL R14, R1, #16
ADD R14, R14, R0
```

```
STR R10, [R14]
```

```
ADD R0, R0, #1
JMP LOOP_X
```

```
END_PROGRAM:
HALT
```

3.3.3 Assembly Test Program

The simulation is driven by a Cocotb-based Python testbench. This allows automated and repeatable verification. The testbench generates the clock and reset signals. It then monitors program execution. The test passes when the global halt signal is asserted. This indicates that all threads have completed execution. Execution traces and waveform dumping are enabled. These support debugging and timing analysis. This verification confirms that the compute core operates correctly within the GPU framework.

```

# testbench/test_matadd_simple.py

import cocotb
from cocotb.triggers import RisingEdge
from cocotb.clock import Clock

@cocotb.test()
async def test_matadd_simple(dut):
    """
    Simplified Testbench for Matrix Addition on GPU Simulation using Cocotb.
    """

    # 1. Create a 10 ns period clock on port clk
    clock = Clock(dut.clk, 10, units="ns") # 10 ns period
    cocotb.start_soon(clock.start())
    dut.reset.value = 1
    cocotb.log.info("Clock started with 10 ns period. Asserting reset.")

    # 2. Initialize reset
    await RisingEdge(dut.clk)
    dut.reset.value = 0
    cocotb.log.info("Deasserting reset.")
    await RisingEdge(dut.clk)

    # 3. Wait for 'halt' signal or timeout after max_cycles
    cocotb.log.info("Waiting for 'halt' signal.")
    max_cycles = 1000 # Maximum number of clock cycles to wait
    cycle = 0

    while dut.halt.value != 1 and cycle < max_cycles:
        await RisingEdge(dut.clk)
        cycle += 1
        if cycle % 100 == 0:
            cocotb.log.info(f"Cycle {cycle}: 'halt' not yet asserted.")

    # 4. Check if 'halt' was asserted
    if dut.halt.value != 1:
        cocotb.log.error(f"Test FAILED: 'halt' signal not asserted after {max_cycles} cycles.")
        assert False, f"'halt' not asserted after {max_cycles} cycles."
    else:
        cocotb.log.info(f"Test PASSED: 'halt' signal asserted after {cycle} cycles.")

```

```
assert True
```

4. Simulation Result

```
tandang@MacBook-Pro-cua-Tan Small_GPU % make SIM=icarus WAVES=1
rm -f results.xml
"/Applications/Xcode.app/Contents/Developer/usr/bin/make" -f Makefile results.xml
/opt/homebrew/bin/iverilog -o sim_build/sim.vvp -s compute_core -g2012 -g2012 -I/Users/tandang/
/Users/tandang/Library/Python/3.9/lib/python/site-packages/cocotb_tools/makefiles/simulators/Mal
rm -f results.xml
COCOTB_TEST_MODULES=testbench.test_execution COCOTB_TESTCASE= COCOTB_TEST_FILTER= COCOTB_TOPLEVEL=
/opt/homebrew/bin/vvp -M /Users/tandang/Library/Python/3.9/lib/python/site-packages/cocotb
    ---ns INFO      gpi                                     ..mbed/gpi_embed.cpp:94   in _embed_init
    ---ns INFO      gpi                                     ../gpi/GpiCommon.cpp:79   in gpi_print_i
    0.00ns INFO      cocotb                                Running on Icarus Verilog version 12.0 (
    0.00ns INFO      cocotb                                Seeding Python random module with 176839
    0.00ns INFO      cocotb                                Initialized cocotb v2.0.1 from /Users/tan
    0.00ns INFO      cocotb.regression                    pytest not found, install it to enable t
    0.00ns INFO      cocotb                                Running tests
    0.00ns INFO      cocotb.regression                    running testbench.test_execution.test_m
        Simplified Testbench for Matrix Addi
    0.00ns WARNING   py.warnings                          /Users/tandang/Small_GPU/testbench/test_
        clock = Clock(dut.clk, 10, units="ns")

    0.00ns INFO      test                                  Clock started with 10 ns period. Asserti
WARNING: /Users/tandang/Small_GPU/src/fetcher.v:17: $readmemh(src/instruction_memory.mem): Too r
Instruction Memory Initialized in Fetcher:
instr_mem[0] = 6000
instr_mem[1] = 6101
instr_mem[2] = 0221
instr_mem[3] = 8210
instr_mem[4] = 1330
instr_mem[5] = 3032
instr_mem[6] = 500b
instr_mem[7] = 2443
instr_mem[8] = 7402
instr_mem[9] = f000
instr_mem[10] = 0000
instr_mem[11] = 6501
instr_mem[12] = 0550
instr_mem[13] = 7503
instr_mem[14] = f000
instr_mem[15] = 0000
Data Memory Initialized:
data_mem[0] = 0003
```

```
data_mem[1] = 0005
data_mem[2] = 0000
data_mem[3] = 0000
data_mem[4] = 0000
data_mem[5] = 0000
data_mem[6] = 0000
data_mem[7] = 0000
data_mem[8] = 0000
data_mem[9] = 0000
data_mem[10] = 0000
data_mem[11] = 0000
data_mem[12] = 0000
data_mem[13] = 0000
data_mem[14] = 0000
data_mem[15] = 0000
```

FST info: dumpfile simulation.vcd opened for output.

FST warning: sim_build/cocotb_iverilog_dump.v:3: \$dumpfile called after \$dumpvars started,
using existing file (simulation.vcd).

FST warning: ignoring signals in previously scanned scope compute_core.

FST warning: ignoring signals in previously scanned scope compute_core.alu_inst.

FST warning: ignoring signals in previously scanned scope compute_core.decoder_inst.

FST warning: ignoring signals in previously scanned scope compute_core.fetcher_inst.

FST warning: ignoring signals in previously scanned scope compute_core.scheduler_inst.

0.00ns INFO test Deasserting reset.

DUT Reset at time 0

10.00ns INFO test Waiting for 'halt' signal.

Time=10000		Thread=0		PC=0		Instruction=6000		Opcode=0110		dest=R0		src1=R0		src2=R0		ir
Time=20000		Thread=0		PC=1		Instruction=6101		Opcode=0110		dest=R1		src1=R0		src2=R1		ir
Time=30000		Thread=1		PC=0		Instruction=6000		Opcode=0110		dest=R0		src1=R0		src2=R0		ir
Time=40000		Thread=2		PC=0		Instruction=6000		Opcode=0110		dest=R0		src1=R0		src2=R0		ir
Time=50000		Thread=3		PC=0		Instruction=6000		Opcode=0110		dest=R0		src1=R0		src2=R0		ir
Time=60000		Thread=4		PC=0		Instruction=6000		Opcode=0110		dest=R0		src1=R0		src2=R0		ir
Time=70000		Thread=5		PC=0		Instruction=6000		Opcode=0110		dest=R0		src1=R0		src2=R0		ir
Time=80000		Thread=6		PC=0		Instruction=6000		Opcode=0110		dest=R0		src1=R0		src2=R0		ir
Time=90000		Thread=7		PC=0		Instruction=6000		Opcode=0110		dest=R0		src1=R0		src2=R0		ir
Time=100000		Thread=0		PC=2		Instruction=0221		Opcode=0000		dest=R2		src1=R2		src2=R1		:
Time=110000		Thread=1		PC=1		Instruction=6101		Opcode=0110		dest=R1		src1=R0		src2=R1		:
Time=120000		Thread=2		PC=1		Instruction=6101		Opcode=0110		dest=R1		src1=R0		src2=R1		:
Time=130000		Thread=3		PC=1		Instruction=6101		Opcode=0110		dest=R1		src1=R0		src2=R1		:
Time=140000		Thread=4		PC=1		Instruction=6101		Opcode=0110		dest=R1		src1=R0		src2=R1		:
Time=150000		Thread=5		PC=1		Instruction=6101		Opcode=0110		dest=R1		src1=R0		src2=R1		:
Time=160000		Thread=6		PC=1		Instruction=6101		Opcode=0110		dest=R1		src1=R0		src2=R1		:
Time=170000		Thread=7		PC=1		Instruction=6101		Opcode=0110		dest=R1		src1=R0		src2=R1		:

Time=180000	Thread=0	PC=3	Instruction=8210	Opcode=1000	dest=R2	src1=R1	src2=R0	:
Time=190000	Thread=1	PC=2	Instruction=0221	Opcode=0000	dest=R2	src1=R2	src2=R1	:
Time=200000	Thread=2	PC=2	Instruction=0221	Opcode=0000	dest=R2	src1=R2	src2=R1	:
Time=210000	Thread=3	PC=2	Instruction=0221	Opcode=0000	dest=R2	src1=R2	src2=R1	:
Time=220000	Thread=4	PC=2	Instruction=0221	Opcode=0000	dest=R2	src1=R2	src2=R1	:
Time=230000	Thread=5	PC=2	Instruction=0221	Opcode=0000	dest=R2	src1=R2	src2=R1	:
Time=240000	Thread=6	PC=2	Instruction=0221	Opcode=0000	dest=R2	src1=R2	src2=R1	:
Time=250000	Thread=7	PC=2	Instruction=0221	Opcode=0000	dest=R2	src1=R2	src2=R1	:
Time=260000	Thread=0	PC=4	Instruction=1330	Opcode=0001	dest=R3	src1=R3	src2=R0	:
Time=270000	Thread=1	PC=3	Instruction=8210	Opcode=1000	dest=R2	src1=R1	src2=R0	:
Time=280000	Thread=2	PC=3	Instruction=8210	Opcode=1000	dest=R2	src1=R1	src2=R0	:
Time=290000	Thread=3	PC=3	Instruction=8210	Opcode=1000	dest=R2	src1=R1	src2=R0	:
Time=300000	Thread=4	PC=3	Instruction=8210	Opcode=1000	dest=R2	src1=R1	src2=R0	:
Time=310000	Thread=5	PC=3	Instruction=8210	Opcode=1000	dest=R2	src1=R1	src2=R0	:
Time=320000	Thread=6	PC=3	Instruction=8210	Opcode=1000	dest=R2	src1=R1	src2=R0	:
Time=330000	Thread=7	PC=3	Instruction=8210	Opcode=1000	dest=R2	src1=R1	src2=R0	:
Time=340000	Thread=0	PC=5	Instruction=3032	Opcode=0011	dest=R0	src1=R3	src2=R2	:
Time=350000	Thread=1	PC=4	Instruction=1330	Opcode=0001	dest=R3	src1=R3	src2=R0	:
Time=360000	Thread=2	PC=4	Instruction=1330	Opcode=0001	dest=R3	src1=R3	src2=R0	:
Time=370000	Thread=3	PC=4	Instruction=1330	Opcode=0001	dest=R3	src1=R3	src2=R0	:
Time=380000	Thread=4	PC=4	Instruction=1330	Opcode=0001	dest=R3	src1=R3	src2=R0	:
Time=390000	Thread=5	PC=4	Instruction=1330	Opcode=0001	dest=R3	src1=R3	src2=R0	:
Time=400000	Thread=6	PC=4	Instruction=1330	Opcode=0001	dest=R3	src1=R3	src2=R0	:
Time=410000	Thread=7	PC=4	Instruction=1330	Opcode=0001	dest=R3	src1=R3	src2=R0	:
Time=420000	Thread=0	PC=6	Instruction=500b	Opcode=0101	dest=R0	src1=R0	src2=R11	:
Time=430000	Thread=1	PC=5	Instruction=3032	Opcode=0011	dest=R0	src1=R3	src2=R2	:
Time=440000	Thread=2	PC=5	Instruction=3032	Opcode=0011	dest=R0	src1=R3	src2=R2	:
Time=450000	Thread=3	PC=5	Instruction=3032	Opcode=0011	dest=R0	src1=R3	src2=R2	:
Time=460000	Thread=4	PC=5	Instruction=3032	Opcode=0011	dest=R0	src1=R3	src2=R2	:
Time=470000	Thread=5	PC=5	Instruction=3032	Opcode=0011	dest=R0	src1=R3	src2=R2	:
Time=480000	Thread=6	PC=5	Instruction=3032	Opcode=0011	dest=R0	src1=R3	src2=R2	:
Time=490000	Thread=7	PC=5	Instruction=3032	Opcode=0011	dest=R0	src1=R3	src2=R2	:
Time=500000	Thread=0	PC=11	Instruction=6501	Opcode=0110	dest=R5	src1=R0	src2=R1	:
Time=510000	Thread=1	PC=6	Instruction=500b	Opcode=0101	dest=R0	src1=R0	src2=R11	:
Time=520000	Thread=2	PC=6	Instruction=500b	Opcode=0101	dest=R0	src1=R0	src2=R11	:
Time=530000	Thread=3	PC=6	Instruction=500b	Opcode=0101	dest=R0	src1=R0	src2=R11	:
Time=540000	Thread=4	PC=6	Instruction=500b	Opcode=0101	dest=R0	src1=R0	src2=R11	:
Time=550000	Thread=5	PC=6	Instruction=500b	Opcode=0101	dest=R0	src1=R0	src2=R11	:
Time=560000	Thread=6	PC=6	Instruction=500b	Opcode=0101	dest=R0	src1=R0	src2=R11	:
Time=570000	Thread=7	PC=6	Instruction=500b	Opcode=0101	dest=R0	src1=R0	src2=R11	:
Time=580000	Thread=0	PC=12	Instruction=0550	Opcode=0000	dest=R5	src1=R5	src2=R0	:
Time=590000	Thread=1	PC=7	Instruction=2443	Opcode=0010	dest=R4	src1=R4	src2=R3	:
Time=600000	Thread=2	PC=7	Instruction=2443	Opcode=0010	dest=R4	src1=R4	src2=R3	:

```

Time=610000 | Thread=3 | PC=7 | Instruction=2443 | Opcode=0010 | dest=R4 | src1=R4 | src2=R3 | :
Time=620000 | Thread=4 | PC=7 | Instruction=2443 | Opcode=0010 | dest=R4 | src1=R4 | src2=R3 | :
Time=630000 | Thread=5 | PC=7 | Instruction=2443 | Opcode=0010 | dest=R4 | src1=R4 | src2=R3 | :
Time=640000 | Thread=6 | PC=7 | Instruction=2443 | Opcode=0010 | dest=R4 | src1=R4 | src2=R3 | :
Time=650000 | Thread=7 | PC=7 | Instruction=2443 | Opcode=0010 | dest=R4 | src1=R4 | src2=R3 | :
Time=660000 | Thread=0 | PC=13 | Instruction=7503 | Opcode=0111 | dest=R5 | src1=R0 | src2=R3 | :
Time=670000 | Thread=1 | PC=8 | Instruction=7402 | Opcode=0111 | dest=R4 | src1=R0 | src2=R2 | :
Time=680000 | Thread=2 | PC=8 | Instruction=7402 | Opcode=0111 | dest=R4 | src1=R0 | src2=R2 | :
Time=690000 | Thread=3 | PC=8 | Instruction=7402 | Opcode=0111 | dest=R4 | src1=R0 | src2=R2 | :
Time=700000 | Thread=4 | PC=8 | Instruction=7402 | Opcode=0111 | dest=R4 | src1=R0 | src2=R2 | :
Time=710000 | Thread=5 | PC=8 | Instruction=7402 | Opcode=0111 | dest=R4 | src1=R0 | src2=R2 | :
Time=720000 | Thread=6 | PC=8 | Instruction=7402 | Opcode=0111 | dest=R4 | src1=R0 | src2=R2 | :
Time=730000 | Thread=7 | PC=8 | Instruction=7402 | Opcode=0111 | dest=R4 | src1=R0 | src2=R2 | :
Thread 0 executing HALT. Halting.
Time=740000 | Thread=0 | PC=14 | Instruction=f000 | Opcode=1111 | dest=R0 | src1=R0 | src2=R0 | :
Thread 1 executing HALT. Halting.
Time=750000 | Thread=1 | PC=9 | Instruction=f000 | Opcode=1111 | dest=R0 | src1=R0 | src2=R0 | :
Thread 2 executing HALT. Halting.
Time=760000 | Thread=2 | PC=9 | Instruction=f000 | Opcode=1111 | dest=R0 | src1=R0 | src2=R0 | :
Thread 3 executing HALT. Halting.
Time=770000 | Thread=3 | PC=9 | Instruction=f000 | Opcode=1111 | dest=R0 | src1=R0 | src2=R0 | :
Thread 4 executing HALT. Halting.
Time=780000 | Thread=4 | PC=9 | Instruction=f000 | Opcode=1111 | dest=R0 | src1=R0 | src2=R0 | :
Thread 5 executing HALT. Halting.
Time=790000 | Thread=5 | PC=9 | Instruction=f000 | Opcode=1111 | dest=R0 | src1=R0 | src2=R0 | :
Thread 6 executing HALT. Halting.
Time=800000 | Thread=6 | PC=9 | Instruction=f000 | Opcode=1111 | dest=R0 | src1=R0 | src2=R0 | :
Thread 7 executing HALT. Halting.
Time=810000 | Thread=7 | PC=9 | Instruction=f000 | Opcode=1111 | dest=R0 | src1=R0 | src2=R0 | :
All threads have halted at time 810000

```

```

820.00ns INFO      test
820.00ns INFO      cocotb.regression
820.00ns INFO      cocotb.regression

```

```

Test PASSED: 'halt' signal asserted after
testbench.test_execution.test_matadd_sir
*****
** TEST
*****
** testbench.test_execution.test_matadd_
*****
** TESTS=1 PASS=1 FAIL=0 SKIP=0
*****

```

Development Progress Documentation

- Week 1: Project initiation, GPU architecture research, Verilog syntax learning, and team task allocation.
- Week 2: Development environment setup, high-level architecture design, initial module specifications, and core module implementation.
- Week 3: Module integration, simulation with predefined programs, debugging, and optimization of system components.

III. Contribution

Member	Task Weight (%)	Details of Contribution
1. Lê Ngô Trác Việt	21.33%	Designed and implemented the instruction decoder (<code>decoder.v</code>) and ALU (<code>alu.v</code>). Conducted an introductory study on GPU architecture and contributed to the overall architectural understanding of the system.
2. Nguyễn Minh Việt	21.33%	Implemented the scheduler module, instruction memory, input memory, and system definitions. Supported the integration of memory-related components.
3. Đặng Hoàng Tân	21.33%	Developed the compute core module, ensuring correct interaction between execution units and other system components.
4. Bùi Xuân Thái	18%	Created and maintained testbenches for system modules, verifying correctness and compatibility during simulation.
5. Vũ Quang Huy	18%	Collaborated on building comprehensive testbenches for all modules, ensuring stable simulation results and functional verification.

IV. Acknowledgements

We would like to express our sincere gratitude to Dr. Nguyen Dinh Han for his invaluable guidance, constructive feedback, and continuous support throughout the development of this project. His expertise in computer architecture and hardware design provided essential direction and greatly contributed to the successful completion of this work.

We also thank our classmates and peers for their helpful discussions, suggestions, and assistance during the design, implementation, and testing phases of the project.

V. Reference

Patterson, D. A., & Hennessy, J. L. (2014). Computer organization and design: The hardware/software interface (5th ed.). Morgan Kaufmann.

Maj, A. (n.d.). Tiny-GPU: A simple GPU design [GitHub repository]. <https://github.com/adam-maj/tiny-gpu>